



รายงานการเขียนโปรแกรม Socket Programming

เรื่อง Reliable File Transfer over UDP

จัดทำโดย

นายศิริสุข	ทานธรรม	รหัสนิสิต	6610402230	หมู่	1
นางสาวนันท์นภัส	ภูริภัทรพันธุ์	รหัสนิสิต	6610450960	หมู่	200
นางสาวรัตนกมล	แก้วมัตย์	รหัสนิสิต	6610451061	หมู่	200

เสนอ

ผศ.ดร. สุขุมาล กิตติสิน

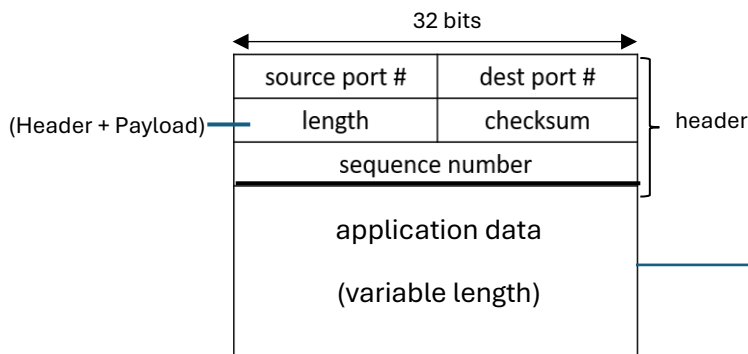
รายงานนี้เป็นส่วนหนึ่งของวิชา 01418351 หลักการสื่อสารคอมพิวเตอร์และการประมวลผลบนคลาวด์

คณะวิทยาศาสตร์ สาขาวิทยาการคอมพิวเตอร์

มหาวิทยาลัยเกษตรศาสตร์

การออกแบบโปรโทคอล เลือกใช้ stop-and-wait

1. Packet format



2. Data Format

data sent by application into UDP socket on a per-byte

```
struct MetaData {  
    int type; 

|         |          |     |     |          |
|---------|----------|-----|-----|----------|
| REQUEST | RESPONSE | ACK | NAK | COMPLETE |
|---------|----------|-----|-----|----------|

  
    char filename[256];  
    bool fileExists; 0 = file not exists , 1 = file exists  
    int fileSize; ขนาดไฟล์ทั้งหมด (byte)  
    int totalSegments; [ fileSize / maxPayloadSize ]  
    int maxPayloadSize; ขนาดสูงสุดของ Payload (Header - length)  
};
```

3. CheckSum แปลงทุกอย่างให้เป็น 16 bits แล้วบวกสะสม

Diagram illustrating the checksum calculation process. The header and payload are converted to 16-bit words and summed. The checksum is calculated as the one's complement of the sum.

```
// กรณี payload มีไบนารีคั่ง 1 ไบนารี (ขนาดเป็นจำนวนคี่)  
// เอาไบนารีคั่งมาวางเป็นไบนารีของค่า 16 บิต จากนั้น shift << 8 แล้วบวกเพิ่ม  
// ทำให้ไบนารีคั่งเป็น 0  
if (payloadSize % 2 == 1) {  
    unsigned char* bytePayload = (unsigned char*)segment->payload;  
    sum += bytePayload[payloadSize - 1] << 8;  
}  
  
// กรณีผลรวมสั้นเกิน 16 บิต ให้เอาค่า ส่วนที่เกิน (บิต 17 ขึ้นไป) มาบวกกลับลง  
// ไปเรื่อย ๆ จนเหลือแค่ 16 บิต ขั้นตอนมาตรฐานของ one's complement sum  
while (sum >> 16) {  
    sum = (sum & 0xFFFF) + (sum >> 16);  
}  
  
// กลับบิตทั้งหมดของผลรวม 16 บิต (one's complement) ได้ checksum  
return ~sum;
```

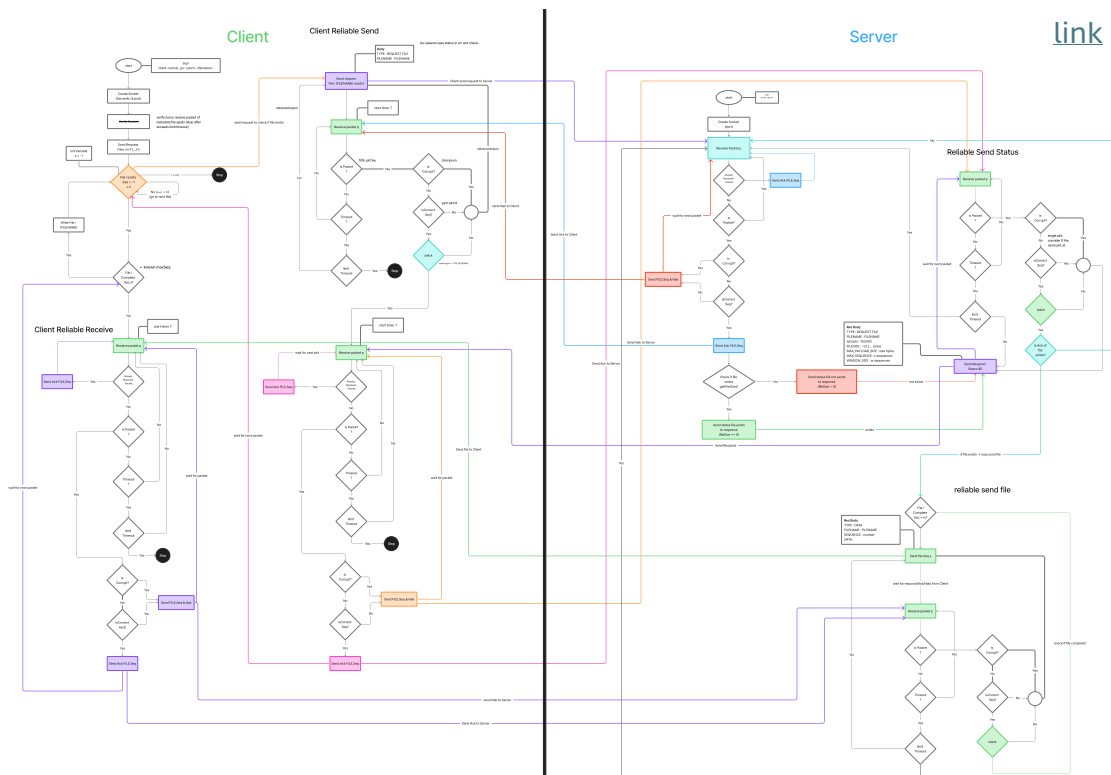
4. Packetization

ฝั่งส่ง (Packetization) แบ่งไฟล์เป็นชิ้นขนาด limitByte (เพื่อหลีกเลี่ยงการชน MTU) โดยกำหนดเลขลำดับ seqNumber = part (เริ่มที่ 0) และคำนวณจำนวนชิ้นทั้งหมดด้วย $\text{int part} = (\text{fileSize} + \text{limitByte} - 1) / \text{limitByte};$ จากนั้นนำข้อมูลสร้าง Segment ที่ละชิ้น โดยใส่ Header (length = HEADER_SIZE + payload_length, seqNumber, checksum) และ Payload (ขนาดไม่เกิน limitByte) และคำนวณ checksum แล้วส่งออกตามลำดับ 0, 1, 2, ..., N-1

ฝั่งรับ (Reassembly) คำนวณความยาวรวมของไฟล์จากขนาด payload ของแต่ละ Segment (ชิ้นทั่วไปมีขนาดไม่เกิน MAX_PAYLOAD_SIZE และชิ้นสุดท้ายมักสั้นกว่า) จากนั้นเรียงตามลำดับ seqNumber แล้วคัดลอก payload ต่อท้ายกันทีละชิ้นจนได้ไฟล์เดิม โดยแต่ละชิ้นต้องตรวจสอบ checksum ก่อนนำมาประกอบรวม

5. File Handler เขียนและอ่านไฟล์ด้วยขนาด 1 Byte ในแต่ละครั้งจนจบขนาดไฟล์

Reliability Mechanism



จากรูปจะมีการแบ่งฝั่งการทำงานทั้งหมด 2 ฝั่งคือ Server และ Client โดยมีการทำงานตามลำดับดังต่อไปนี้

Reliable Send เป็นการส่งและรอรับ Ack เพื่อเช็คว่าได้รับแล้วหากหมด Timeout จะส่งกลับไปใหม่จนหมด Max Retries และจะจบการทำงานในการส่ง หากได้รับ Nak จะส่งข้อกลับไปใหม่อีกครั้ง

1. Server เริ่มต้นการทำงานด้วยการรอรับ Request Packet จาก Client ตามรูปแบบ Reliable Data Transfer โดยใช้หลักการ Ack และ Nak
 - 1.1. เริ่มต้นเมื่อ Server รับ Packet จะเช็คว่าเป็นประเภท Request, Ack หรือ Nak
 - 1.2. Packet ทุกประเภทเมื่อได้รับจะถูกดำเนินการ Checksum และส่ง Nak เมื่อ Corrupt
 - 1.3. เมื่อได้รับ Request File จะส่ง Ack กลับไปหา Client และเช็คว่ามีไฟล์หรือไม่ และทำการ Reliable Send ไฟล์ชนิด MetaData ด้วย TYPE_RESPONSE กลับไปหา client เพื่อบอกสถานะและรอรับ Ack และส่งใหม่หาก timeout จนจบ Max Retries หากไม่ได้รับการตอบรับจะหยุดการดำเนินการของ Request File นั้น
 - 1.4. เมื่อได้รับ Ack ว่า Server รับรู้ MetaData เรียบร้อยแล้วจะเริ่มส่ง Packet ตามจำนวน Sequence ที่ทำการ Packetization ด้วย Reliable Send Mechanism
 - 1.4.1. เมื่อจบการส่งจะส่งบอก Client ด้วย MetaData แบบ TYPE_COMPLETE และรอรับ Ack ตาม RDT (จบการทำงาน)
 - 1.5. การ Handle Ack เมื่อรับ Ack จะเพิ่มค่า Sequence ขึ้นทุกครั้งที่ได้รับ Ack ที่ตรงกันเพื่อส่งต่อใน Sequence ลำดับถัดไป
 - 1.5.1. หากไม่ใช่ Sequence ที่ต้องการจะไม่สนใจ Packet นั้น
 - 1.5.2. หากได้รับไฟล์ที่ไม่ตรงกับ Sequence ที่ต้องการได้รับแล้ว จะส่ง Ack กลับไปบอก Client
 - 1.6. การ Handle Nak จะส่ง Sequence ปัจจุบันกลับไปใหม่

Reliable Receive เป็นการรับแพ็กเก็ต ตรวจสอบ Checksum ใช้ expected_seq เป็นแกนควบคุมลำดับ Ack เมื่อถูกต้อง

หรือ ได้รับซ้ำ Nak เมื่อขาดลำดับ หรือ Corrupt และประกอบไฟล์จนครบแล้วค่อยรับ COMPLETE จากนั้นจัดการลำดับและประกอบข้อมูล

2. Client สร้าง Request Packet (TYPE_REQUEST) พร้อมชื่อไฟล์ และรอรับ packet ถ้าไม่พบไฟล์จบของคำขอนี้ และส่งคำขอไฟล์ถัดไป ถ้าข้อมูล MetaData (TYPE_RESPONSE) ถูกต้องและผ่าน Checksum ส่ง Ack กลับ และเตรียมข้อมูล เช่น คำนวณจำนวนชิ้น ขนาดรวม และบัพเฟอร์เขียนไฟล์ สำหรับ Reassembly

2.1. ช่วงรับข้อมูลไฟล์ กำหนด $expected_seq = 0$ เพื่อรอแพ็กเก็ตข้อมูลชิ้นแรก เมื่อรับแพ็กเก็ตข้อมูลแต่ละชิ้น ตรวจสอบ Checksum ถ้า Corrupt ส่ง Nak ของ $expected_seq$ เดิม

2.1.1. ถ้า $seq == expected_seq$ และไม่ Corrupt คัดลอก payload ต่อท้ายในตำแหน่งที่ถูกต้อง ส่ง Ack กลับ

2.1.2. ถ้า $seq < expected_seq$ คือ ได้รับแล้ว ส่ง Ack ของลำดับล่าสุด

2.1.3. ถ้า $seq > expected_seq$ คือ ข้ามลำดับ ส่ง Nak เพื่อขอส่งซ้ำชิ้นที่ขาด

2.2. ปิดการรับ เมื่อ Server ส่ง TYPE_COMPLETE มาและตรวจสอบ Checksum จากนั้น Client ตรวจสอบว่ารับครบตามจำนวนแพ็กเก็ต และขนาดรวมตรงตาม MetaData หากไม่ครบ Client ไม่ยอมรับ COMPLETE แล้วส่ง Nak ของ $expected_seq$ เพื่อให้ Server ส่งซ้ำที่ขาด ก่อนจะยอมรับ COMPLETE อีกครั้ง

3. **วิธีการจำลอง error** สุ่ม drop และ corrupt packet เป็นเปอร์เซ็นต์ใน client และ/หรือ server

Client Loss %	Client Corrupt%	Server Loss %	Server Corrupt%	Client and Server Max Retry	Total time (second)		Pass/Fail	
					1 File	5 Files	1 File	5 Files
0	0	0	0	100 k	0.324	0.547	Pass	
90	0	0	0		53.020	95.620		
0	90	0	0		1.718	2.944		
30	30	0	0		1.610	2.704		
0	0	50	0		11.443	21.413		
0	0	0	50		1.294	1.294		
0	0	25	25		2.558	2.558		
10	10	10	10		0.743	1.407		
20	20	20	20		4.089	8.467		

ข้อจำกัดและสิ่งที่ควรปรับปรุงในอนาคต

ตามโปรโตคอล Client ส่ง Metadata เป็น TYPE_REQUEST ให้ Server และ Server รับคำขอ และส่ง Ack จากนั้น Server ไปทำการเตรียมส่งไฟล์และส่ง TYPE_RESPONSE แต่ Ack นั้น ถูก drop หรือ corrupt ทำให้ Client ไม่ได้รับ Ack และ Client ส่ง TYPE_REQUEST ใหม่ แต่ Server ต้องการรับ ACK จาก Reliable Send ทำให้วนลูบจนครบ maxRetries แล้ว Server กลับไปรับ Request File จึงทำให้ใช้เวลานานในการส่ง